

Reversal Pattern

1. Reverse Linked List (Iterative)

LeetCode 206

Concept

- Maintain `prev`, `curr`, `next`.
- Store `next` before changing links.
- Reverse: `curr->next = prev`.
- Move all pointers one step.
- `prev` becomes the new head.

Edge Cases

- Empty list.
- Single node.

Tips

- Always save `next` first.
- **Time:** `O(n)` **Space:** `O(1)`.

```
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *curr = head;

    while (curr) {
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }

    return prev;
}
```

2. Reverse Linked List (Recursive)

LeetCode 206

Concept

- Reverse the remaining list first.
- Attach current node to the end:
 - `head->next->next = head`
 - `head->next = nullptr`
- Base case returns the new head.

Edge Cases

- Empty list.
- Single node (base case).

Tips

- Remember to set `head->next = nullptr` to avoid cycles.
- **Time:** `O(n)` **Space:** `O(n)` (recursion stack).

```
ListNode* reverseList(ListNode* head) {
    if (!head || !head->next)
        return head;

    ListNode* newHead = reverseList(head->next);

    head->next->next = head;
    head->next = nullptr;

    return newHead;
}
```

3. Reverse Linked List II (Must Do)*****

LeetCode 92

Concept

- Use a **dummy node** to handle `left = 1`.
- Reach the node **before** position `left`.
- Reverse nodes from `left` to `right` using **head insertion**.
- Reconnect automatically during reversal.

Edge Cases

- `left == right` (no reversal).
- `left == 1` (dummy handles it).
- Reverse till the last node.

EXTENDING THE SAME IDEA FOR AN EFFICIENT SOLUTION

(right - left + 1)

left = 3
right = 5

```
ListNode preNode = null;
ListNode currNode = head;
for (3 times)
while (currNode != null) {
    ListNode nextNode = currNode.next;
    currNode.next = preNode;
    preNode = currNode;
    currNode = nextNode;
}
```

DRY-RUN

```
// create a dummy node to mark the head of this list
ListNode dummy = new ListNode(0);
dummy.next = head;

// make markers for currentNode and for the node before reversing
ListNode leftPre = dummy;
ListNode currNode = head;

for (int i = 0; i < left - 1; i++) {
    leftPre = leftPre.next;
    currNode = currNode.next;
}

// make a marker to node where we start reversing
ListNode subListHead = currNode;

ListNode preNode = null;
for (int i = 0; i <= right - left; i++) {
    ListNode nextNode = currNode.next;
    currNode.next = preNode;
    preNode = currNode;
    currNode = nextNode;
}

// Join the pieces
leftPre.next = preNode;
subListHead.next = currNode;

return dummy.next;
```

4. Reverse Nodes in k-Group (Must Do)*****

Cpp ^

Copy

```
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        // Step 1: check if there are at least k nodes
        ListNode* node = head;
        for (int i = 0; i < k; i++) {
            if (!node) return head; // fewer than k nodes, return as-is
            node = node->next;
        }

        // Step 2: reverse k nodes
        ListNode* prev = nullptr;
        ListNode* curr = head;
        for (int i = 0; i < k; i++) {
            ListNode* next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }

        // Step 3: recurse on the remaining list
        head->next = reverseKGroup(curr, k);

        // Step 4: return new head of this group
        return prev;
    }
};
```

5. Swap Nodes in Pairs (just k=2)

LeetCode 24

Concept

- Reverse every group of 2

Excellent introduction before K-group reversal.

Difficulty: Medium

Interview Frequency: ★★☆☆☆

6. Reverse Even Length Groups*****

LeetCode 2074

Concept

- Split into groups
- Reverse only even-sized groups

Difficulty: Medium

```

ListNode* reverseKGroup(ListNode* head, int k) {
    if(!head) return head;

    int n=1;
    ListNode * temp = head;
    while(temp->next){
        temp = temp->next;
        n++;
    }
    // just counted nodes till now

    // check if number of nodes >= k else return head;
    int num = k;
    if(n<k) num = n;

    if(num&1){
        ListNode* curr = head , *prev =NULL;

        for(int i=0;i<num;i++){
            prev = curr;
            curr = curr->next;
        }

        prev->next = reverseKGroup(curr , num+1);

        return head;
    }

    ListNode * curr = head , *prev = NULL , *next = NULL;

    // k-1 times
    for(int i=0;i<num;i++){
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    head->next = reverseKGroup(curr , num+1);

    return prev;
}

ListNode* reverseEvenLengthGroups(ListNode* head) {
    return reverseKGroup(head , 1);
}

```